

Started on Tuesday, 28 April 2026, 10:29 AM

State Finished

Completed on Tuesday, 28 April 2026, 11:02 AM

Time taken 33 mins 6 secs

Grade 9.00 out of 10.00 (90%)

Question 1 | Correct Mark 1.00 out of 1.00

Balanced strings are those that have an equal quantity of 'L' and 'R' characters.

Given a balanced string *s*, split it in the maximum amount of balanced strings.

Return the maximum amount of split balanced strings.

Example 1:

Input:

RLRRLRLRL

Output:

4

Explanation: *s* can be split into "RL", "RRL", "RL", "RL", each substring contains same number of 'L' and 'R'.

Example 2:

Input:

RLLLLRRRLR

Output:

3

Explanation: *s* can be split into "RL", "LLLLRRR", "LR", each substring contains same number of 'L' and 'R'.

Example 3:

Input:

LLLLRRRR

Output:

1

Explanation: *s* can be split into "LLLLRRRR".

Constraints:

$1 \leq s.length \leq 1000$

s[*i*] is either 'L' or 'R'.

s is a balanced string.

For example:

Test	Result
<code>print(BalancedStrings('RLRRLRLRL'))</code>	4
<code>print(BalancedStrings('RLLLLRRRLR'))</code>	3

Answer: (penalty regime: 0 %)

Reset answer

```
1 def BalancedStrings(s):
2     count = 0
3     balance = 0
4     for char in s:
5         if char == 'R':
6             balance += 1
7         else:
8             balance -= 1
```

```
9 |         if balance ==0:
10 |             count+=1
11 |     return count
12 |
```

	Test	Expected	Got	
✓	print(BalancedStrings('RLRLLRLRL'))	4	4	✓
✓	print(BalancedStrings('RLLLLRRRLR'))	3	3	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 2 | Correct Mark 1.00 out of 1.00

Write a Python program for binary search.

For example:

Input	Result
1,2,3,5,8 6	False
3,5,9,45,42 42	True

Answer: (penalty regime: 0 %)

```

1 def BinarySearch(arr,target):
2     left=0
3     right=len(arr)-1
4     while left <right:
5         mid=(left + right)//2
6         if arr[mid]==target:
7             return True
8         elif arr[mid]<target:
9             left=mid+1
10        else:
11            right =mid-1
12        return False
13 arr=list(map(int,input().split(',')))
14 target=int(input())
15 arr.sort()
16 print(BinarySearch(arr,target))
17

```

	Input	Expected	Got	
✓	1,2,3,5,8 6	False	False	✓
✓	3,5,9,45,42 42	True	True	✓
✓	52,45,89,43,11 11	True	True	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 3 | Correct Mark 1.00 out of 1.00

Given an list, find peak element in it. A peak element is an element that is greater than its neighbors.

An element $a[i]$ is a peak element if

$A[i-1] \leq A[i] \geq a[i+1]$ for middle elements. $[0 < i < n-1]$

$A[i-1] \leq A[i]$ for last element $[i=n-1]$

$A[i] \geq A[i+1]$ for first element $[i=0]$

Input Format

The first line contains a single integer n , the length of A .

The second line contains n space-separated integers, $A[i]$.

Output Format

Print peak numbers separated by space.

Sample Input

5

8 9 10 2 6

Sample Output

10 6

For example:

Input	Result
4	12 8
12 3 6 8	

Answer: (penalty regime: 0 %)

```

1 n=int(input())
2 A=list(map(int,input().split()))
3 peaks=[]
4 for i in range(n):
5     if(i==0 and A[i]>A[i+1]) or \
6        (i==n-1 and A[i]>A[i-1]) or \
7        (0<i<n-1 and A[i] >A[i+1] and A[i] >A[i-1]):
8         peaks.append(A[i])
9 print(*peaks)
```

	Input	Expected	Got	
✓	7 15 7 10 8 9 4 6	15 10 9 6	15 10 9 6	✓
✓	4 12 3 6 8	12 8	12 8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 4 | Correct Mark 1.00 out of 1.00

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return *the only number in the range that is missing from the array*.

Example 1:

Input: `nums = [3,0,1]`

Output: 2

Explanation: `n = 3` since there are 3 numbers, so all numbers are in the range `[0,3]`. 2 is the missing number in the range since it does not appear in `nums`.

Example 2:

Input: `nums = [0,1]`

Output: 2

Explanation: `n = 2` since there are 2 numbers, so all numbers are in the range `[0,2]`. 2 is the missing number in the range since it does not appear in `nums`.

Example 3:

Input: `nums = [9,6,4,2,3,5,7,0,1]`

Output: 8

Explanation: `n = 9` since there are 9 numbers, so all numbers are in the range `[0,9]`. 8 is the missing number in the range since it does not appear in `nums`.

For example:

Test	Result
<code>print(missingNumber([3,0,1]))</code>	2
<code>print(missingNumber([0,1]))</code>	2

Answer: (penalty regime: 0 %)

Reset answer

```

1 def missingNumber(numbers):
2     n=len(numbers)
3     total=n*(n+1)//2
4     return total - sum(numbers)
5
6

```

	Test	Expected	Got	
✓	print(missingNumber([3,0,1]))	2	2	✓
✓	print(missingNumber([0,1]))	2	2	✓
✓	print(missingNumber([9,6,4,2,3,5,7,0,1]))	8	8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 5 | Correct Mark 1.00 out of 1.00

You are given an $m \times n$ integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `True` if `target` is in `matrix` or `False` otherwise.

You must write a solution in $O(\log(m * n))$ time complexity.

Example 1:

1	3	5	7
10	11	16	20
23	30	34	60

Input: `matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, `target = 3`

Output: `True`

Example 2:

1	3	5	7
10	11	16	20
23	30	34	60

Input: `matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, `target = 13`

Output: `False`

For example:

Test	Result
<code>print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 13))</code>	False
<code>print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 3))</code>	True

Answer: (penalty regime: 0 %)

Reset answer

```

1 def searchMatrix(matrix, target):
2     for row in matrix:
3         if target in row:
4             return True
5     return False

```

0 |

	Test	Expected	Got	
✓	print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 13))	False	False	✓
✓	print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 3))	True	True	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 6 | Incorrect Mark 0.00 out of 1.00

String should contain only the words are not palindrome.

Sample Input 1

Malayalam is my mother tongue

Sample Output 1

is my mother tongue

Answer: (penalty regime: 0 %)

```

1 s=input().split()
2 result=""
3 for word in s:
4     if word.lower()!=word.lower()[::-1]:
5         result+=word+" "
6 print("".join(result))

```

	Input	Expected	Got	
✓	Malayalam is my mother tongue	is my mother tongue	is my mother tongue	✓

Your code failed one or more hidden tests.

Your code must pass all tests to earn any marks. Try again.

Incorrect

Marks for this submission: 0.00/1.00.

Question 7 | Correct Mark 1.00 out of 1.00

Given two Strings s1 and s2, remove all the characters from s1 which is present in s2.

Constraints

1<= string length <= 200

Sample Input 1

experience
enc

Sample Output 1

xpri

Answer: (penalty regime: 0 %)

```

1 s1=input()
2 s2=input()
3 result=""
4 for letter in s1:
5     if letter not in s2:
6         result+=letter
7 print(result)

```

	Input	Expected	Got	
✓	experience enc	xpri	xpri	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 8 | Correct Mark 1.00 out of 1.00

An list contains N numbers and you want to determine whether two of the numbers sum to a given number K. For example, if the input is 8, 4, 1, 6 and K is 10, the answer is yes (4 and 6). A number may be used twice.

Input Format

The first line contains a single integer n , the length of list

The second line contains n space-separated integers, list[i].

The third line contains integer k.

Output Format

Print Yes or No.

Sample Input

```
7
0 1 2 4 6 5 3
1
```

Sample Output

Yes

For example:

Input	Result
5 8 9 12 15 3 11	Yes
6 2 9 21 32 43 43 1 4	No

Answer: (penalty regime: 0 %)

```
1 n=int(input())
2 numbers = list(map(int,input().split()))
3 k=int(input())
4 found= False
5 for i in range(n):
6     for j in range(i+1,n):
7         if numbers[i]+numbers[j]==k:
8             found=True
9 if found:
10     print("Yes")
11 else:
12     print("No")
13
```

	Input	Expected	Got	
✓	5 8 9 12 15 3 11	Yes	Yes	✓
✓	6 2 9 21 32 43 43 1 4	No	No	✓
✓	6 13 42 31 4 8 9 17	Yes	Yes	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 9 | Correct Mark 1.00 out of 1.00

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`.

If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [-1,0,3,5,9,12]`, `target = 9`

Output: `4`

Explanation: 9 exists in `nums` and its index is 4

Example 2:

Input: `nums = [-1,0,3,5,9,12]`, `target = 2`

Output: `-1`

Explanation: 2 does not exist in `nums` so return -1

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 < \text{nums}[i], \text{target} < 10^4$
- All the integers in `nums` are **unique**.
- `nums` is sorted in ascending order.

For example:

Test	Result
<code>print(search([-1,0,3,5,9,12],9))</code>	4

Answer: (penalty regime: 0 %)

Reset answer

```

1 from typing import List
2 def search(nums:List[int],target:int)-> int:
3     left=0
4     right=len(nums)-1
5     while left <=right:
6         mid=(left+right)//2
7         if nums[mid]==target:
8             return mid
9         elif nums[mid] <target:
10            left = mid +1
11        else:
12            right=mid -1
13    return -1
14
15

```

	Test	Expected	Got	
✓	print(search([-1,0,3,5,9,12],9))	4	4	✓
✓	print(search([-1,0,3,5,9,12],2))	-1	-1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 10 | Correct Mark 1.00 out of 1.00

Two string values S1, S2 are passed as the input. The program must print first N characters present in S1 which are also present in S2.

Input Format:

The first line contains S1.

The second line contains S2.

The third line contains N.

Output Format:

The first line contains the N characters present in S1 which are also present in S2.

Boundary Conditions:

$2 \leq N \leq 10$

$2 \leq \text{Length of S1, S2} \leq 1000$

Example Input/Output 1:

Input:

```
abcbde
cdefghbb
3
```

Output:

```
bcd
```

Note:

b occurs twice in common but must be printed only once.

Answer: (penalty regime: 0 %)

```
1 s1=input()
2 s2=input()
3 n=int(input())
4 result=""
5 for char in s1:
6     if char in s2 and char not in result:
7         result+=char
8     if len(result)==n:
9         break
10 print(result)
```

	Input	Expected	Got	
✓	abcbde cdefghbb 3	bcd	bcd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.